World Models

Petra Galuscakova

petra.galuscakova@uis.no

University of Stavanger

Information Access & Artificial Intelligence

Department of Electrical Engineering and Computer Science
University of Stavanger

March 19, 2026

Overview

Outline

Why World Models?

- Many AI systems must act in dynamic environments.
  - autonomous vehicles
  - robotics
  - game-playing agents
  - virtual assistants
- Key question: How can an AI anticipate the consequences of its actions?
- Drawn by research in AGI and realistic videos (https://openai.com/sora/)
- Generally, world models are regarded as tools for either understanding the present state of the world or predicting its future dynamics [1]

[1] https://arxiv.org/pdf/2411.14499

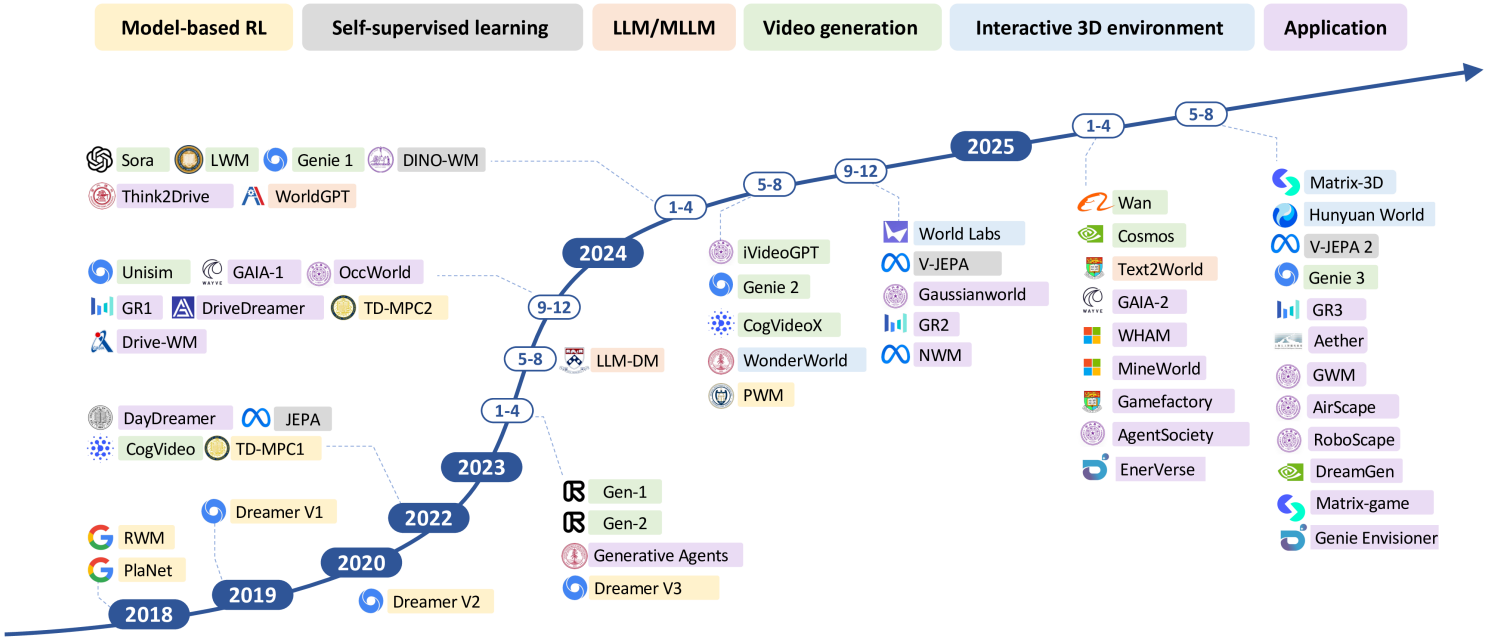
Why World Models?

- While vision models focus on learning abstract, compressed representations of spatial information at a single point in time, world models extend this capability by learning the temporal dynamics of an environment to predict future states
- World models enable deliberate planning by allowing an agent to explore alternative courses of action and assess their likely outcomes before executing them. Standard models that lack this internal predictive model often struggle with long-horizon tasks that require anticipating future states.
- While vision models extract "what" is in a frame, world models learn the "laws of physics" and dynamics of a system to simulate what happens next.

Humans Use World Models

- Humans perform mental simulation during decision making
- Example: Driving a car
- Before overtaking another car you predict speed, distance and reaction time
- This is essentially a world model.
- A world model can be defined as a learned model that predicts how the environment evolves given the current state and actions.

World Model Roadmap[2]



[2] https://arxiv.org/pdf/2411.14499

Pre-neural Models

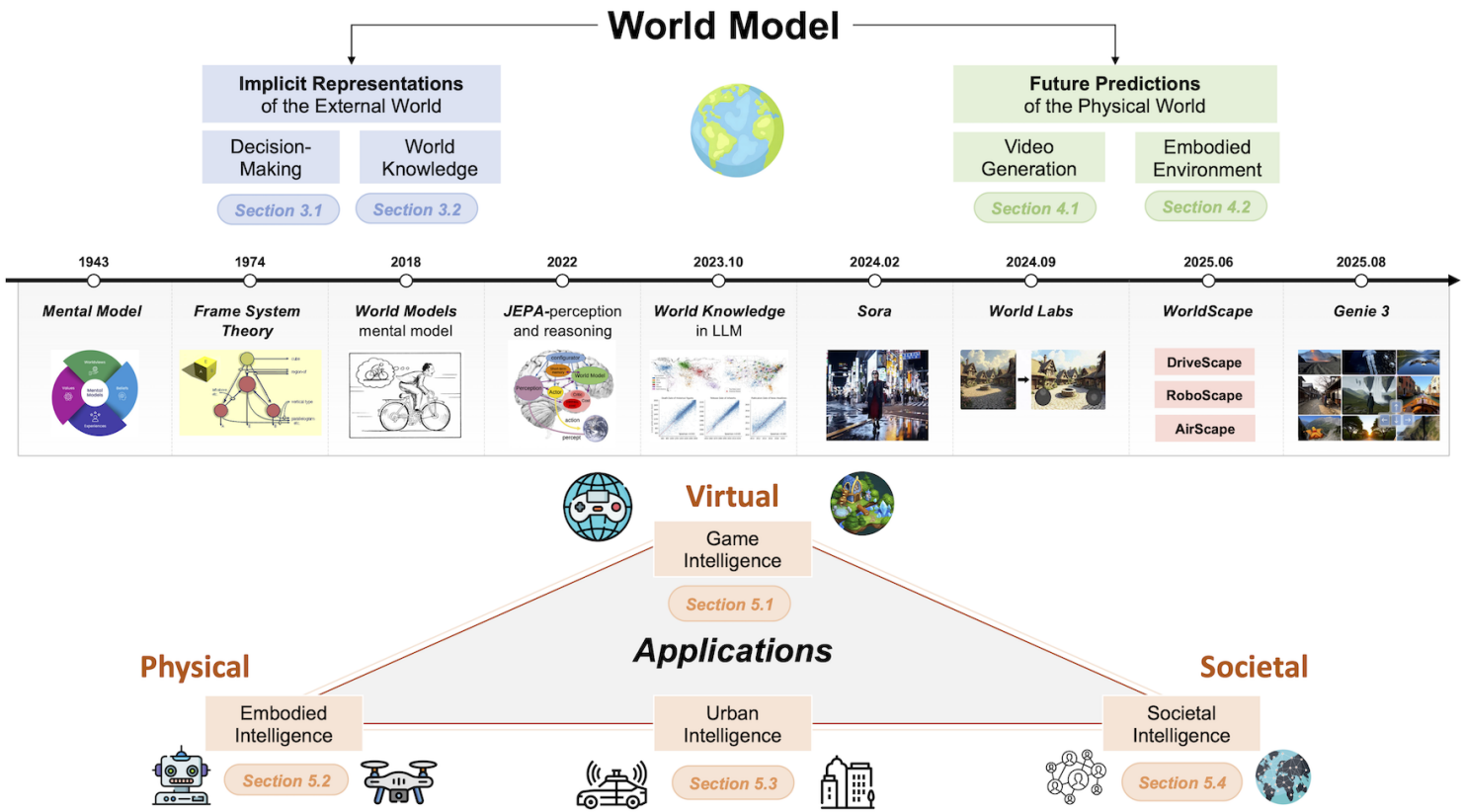
- **Classical Foundation (Minsky, 1974):** Proposed "frames" as data structures to represent stereotyped situations, using "slots" to store specific attributes and expectations.
- Frame theory proposed that humans represent knowledge using data structures called "frames" to understand situations (like entering a room)
- Example – "Restaurant Frame":
  - Slots: customer, waiter, order, food, bill
  - Defaults: order $\rightarrow$ eat $\rightarrow$ pay
  - $\rightarrow$ "She left a tip" $\rightarrow$ infer meal completed, bill paid

World Model Definition[3]

The definition is generally divided into two primary perspectives: understanding the world and predicting the future.

- Ha and Schmidhuber focused on abstracting the external world to gain a deep understanding of its underlying mechanisms.
- In contrast, LeCun argued that a world model should not only perceive and model the real world but also possess the capacity to envision possible future states to inform decision-making.
- Video generation models such as Sora represent an approach that concentrates on simulating future world evolution and thus align more closely with the predictive aspect of world models.

World Model Definition[4] (cont.)

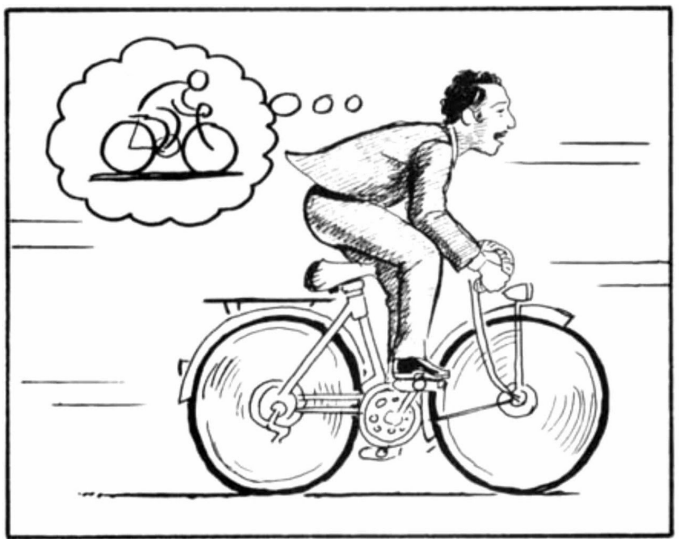


[3] https://arxiv.org/pdf/2411.14499
[4] https://arxiv.org/pdf/2411.14499

Outline

Introduction: Mental Models[5]

- Humans develop an **internal mental model** of the world based on limited sensory input.
- We use these models to make decisions and predict the future, often subconsciously (e.g., hitting a 100 mph fastball).
- The authors explore building generative neural networks to act as these "World Models" for artificial agents.



[5] https://arxiv.org/pdf/1803.10122

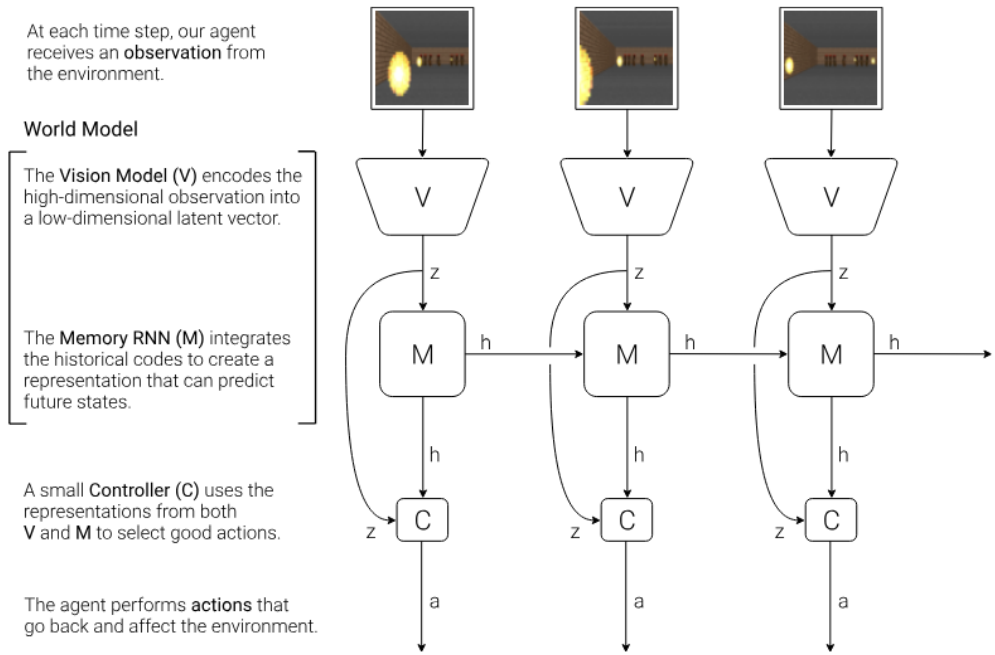
Agent Architecture: V, M, and C

The agent is divided into three components:

Vision (V): Compresses high-dimensional sensory input into a small representation

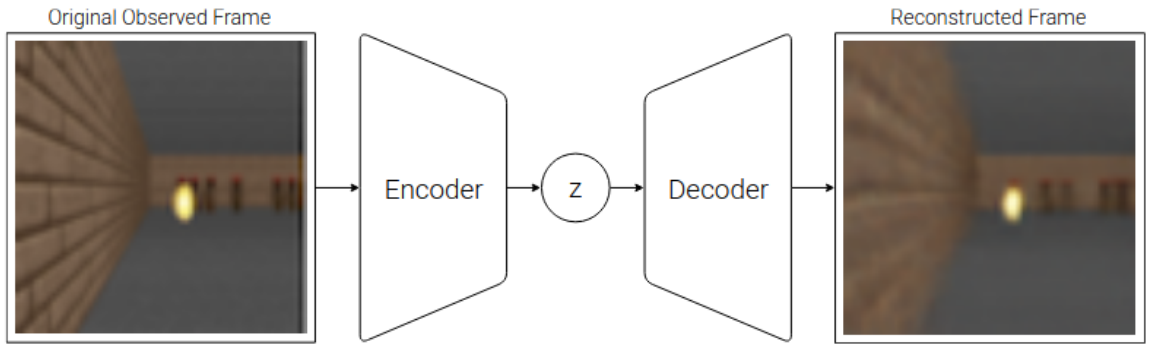
Memory (M): Predicts future codes based on historical information

Controller (C): Makes decisions based only on the representations from V and M



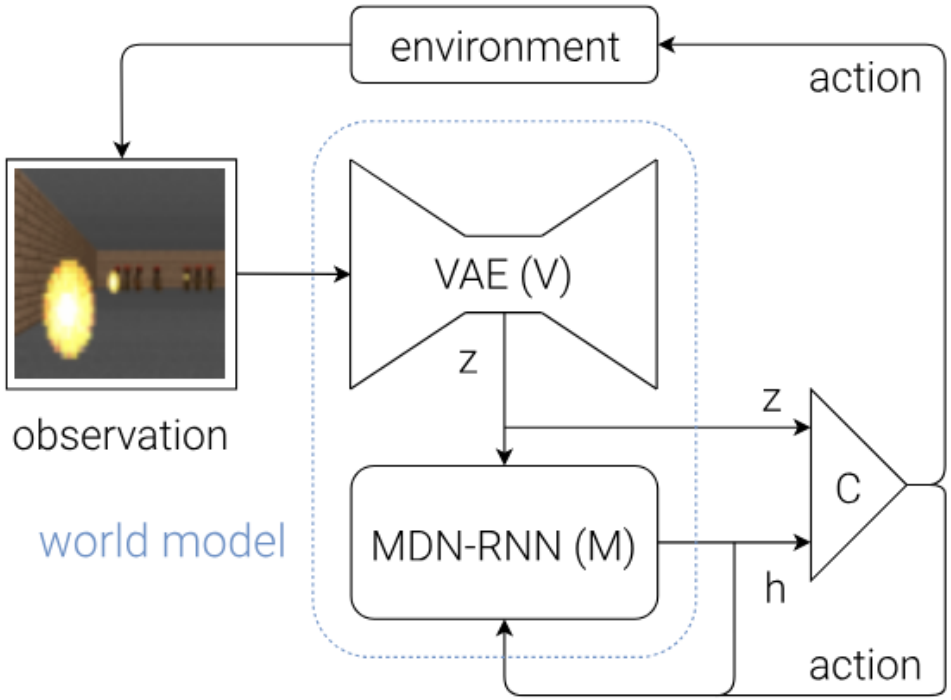
Vision Model (V): Spatial Compression

- Uses a **Variational Autoencoder (VAE)** to compress image frames into a latent vector z
- This allows the agent to work with an abstract, low-dimensional representation of the environment rather than raw pixels



Memory Model (M): Temporal Prediction

- Uses an **MDN-RNN** (LSTM with a Mixture Density Network output) to model the probability distribution of the next latent vector: $P(z_{t+1}|a_t, z_t, h_t)$
- This captures the dynamics of the environment, including stochasticity

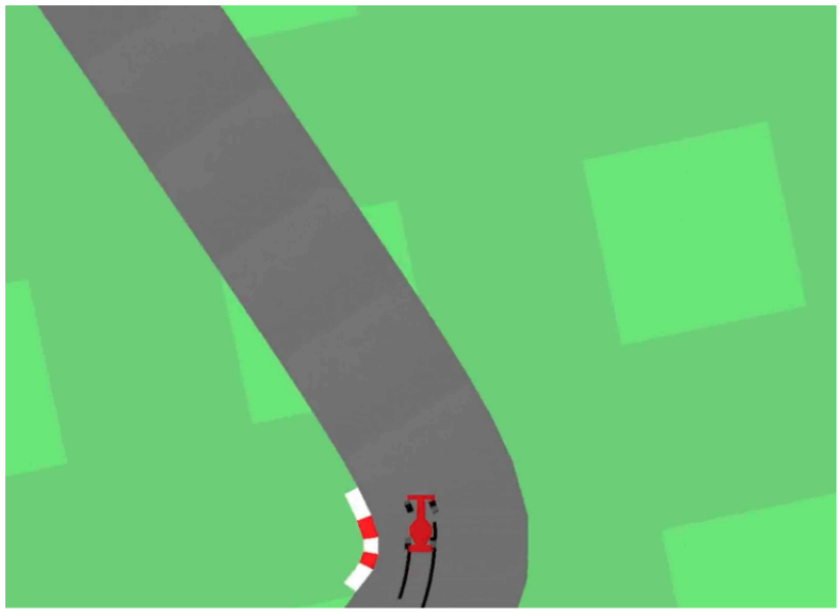


Controller (C): Simple & Compact

- A simple, single-layer linear model: $a_t = W_c[z_t h_t] + b_c$
- By keeping C small, the training algorithm (CMA-ES) focuses on the credit assignment problem on a small search space
- Complexity resides in the world model (V and M), while C provides reflexive behavior

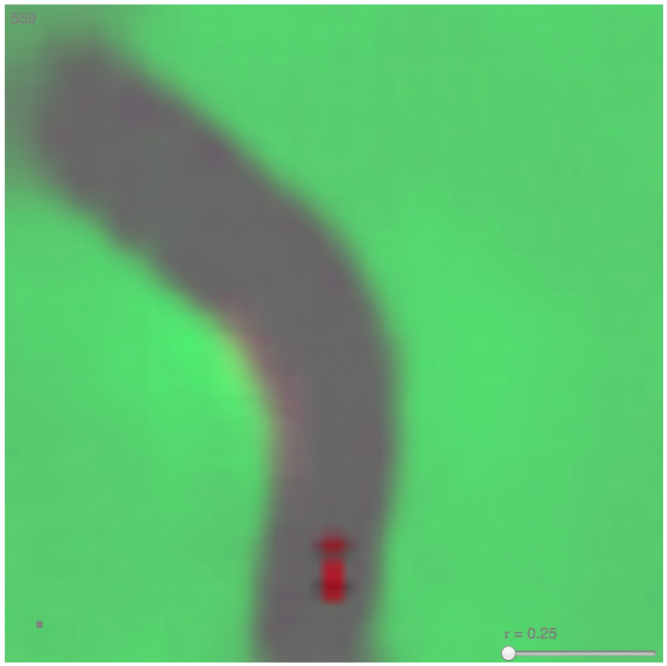
Case Study: Car Racing

- The full world model (V and M) achieved a state-of-the-art score, compared to CNNs
- Access to both z_t (current vision) and h_t (memory/prediction) is crucial for stable driving and handling sharp corners
- The agent "instinctively" predicts the track without needing to plan out explicit future scenarios



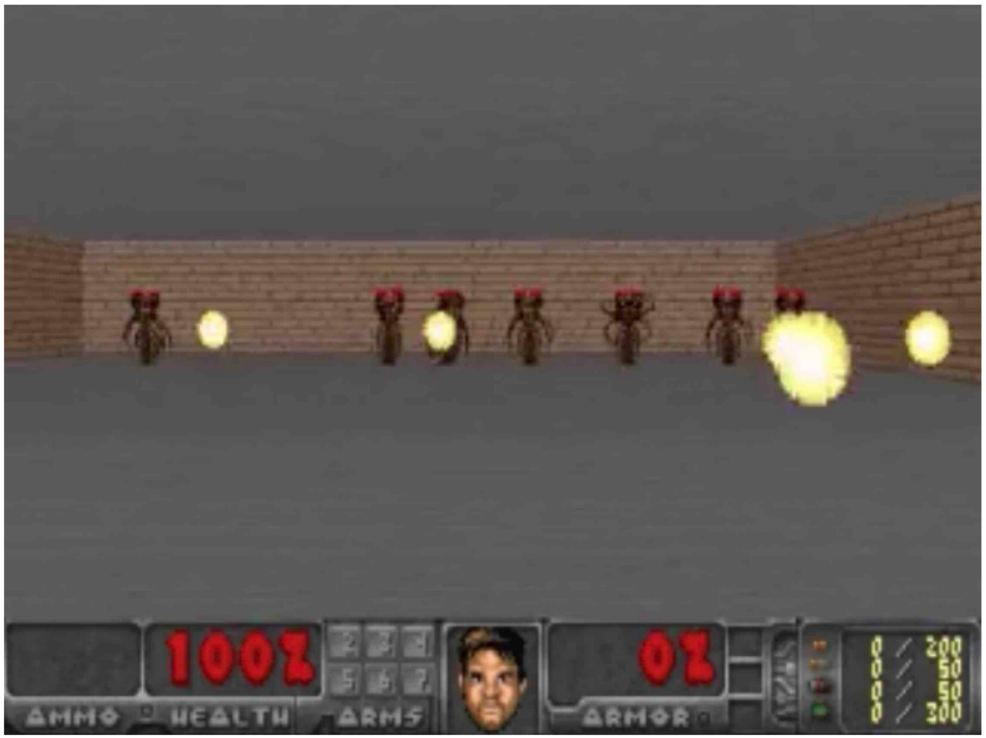
Dream Training: Learning in Hallucinations

- Model can be also used for a generation
- If the world model is accurate, we can substitute the real environment with the modeled one
- **"Dream Training"**: The agent learns entirely inside its own hallucinated environment generated by the MDN-RNN
- This allows for massive parallelization and avoids using heavy compute resources for rendering actual images



VizDoom: Surviving the Nightmare

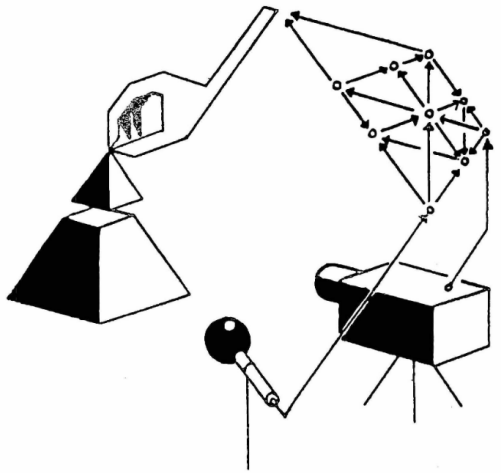
- Agent trained to avoid fireballs **entirely in a latent space dream**
- The model learns game logic, physics, and enemy behavior from raw data



Iterative Training for Complex Worlds

For sophisticated environments, the sources propose a **continual learning loop**:

1. Initialize M and C with random parameters.
2. Perform *N* rollouts in the actual environment to collect data.
3. Train M to model the observed dynamics and train C to optimize reward **inside** M.
4. Repeat the process to explore new areas made available by improved policies.



Intrinsic Motivation and Curiosity

- The MDN-RNN's training loss represents how well the agent understands the world's dynamics
- By **flipping the sign of the loss function**, the agent can be encouraged to seek out unfamiliar states (Artificial Curiosity)
- This encourages exploration of "surprising" parts of the environment to improve the world model's predictive accuracy

Dreamer[6]

- **Dreamer** is an agent that solves complex, long-horizon tasks purely through latent imagination
- It builds a **World Model** from past experience and learns behaviors by predicting hypothetical trajectories in a compact latent space
- World Models paper treats the "dream" environment as a black box; it only requires the final cumulative reward from a rollout to evolve the controller's parameters, rather than the entire gradient history.
- In contrast, Dreamer treats the world model as a differentiable environment, which allows it to use analytic gradients for significantly more efficient learning

[6] https://arxiv.org/pdf/1912.01603

Dreamer[7]

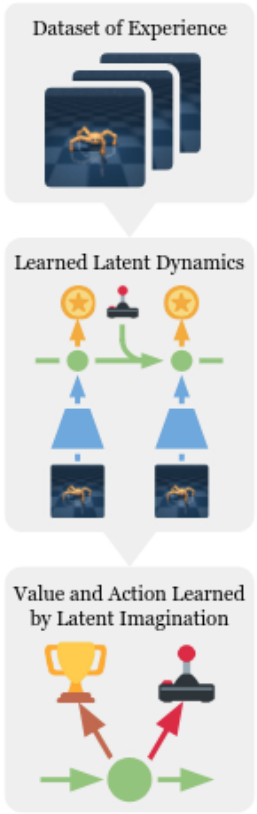


Figure 1: Dreamer learns a world model from past experience and efficiently learns farsighted behaviors in its latent space by backpropagating value estimates back through imagined trajectories.

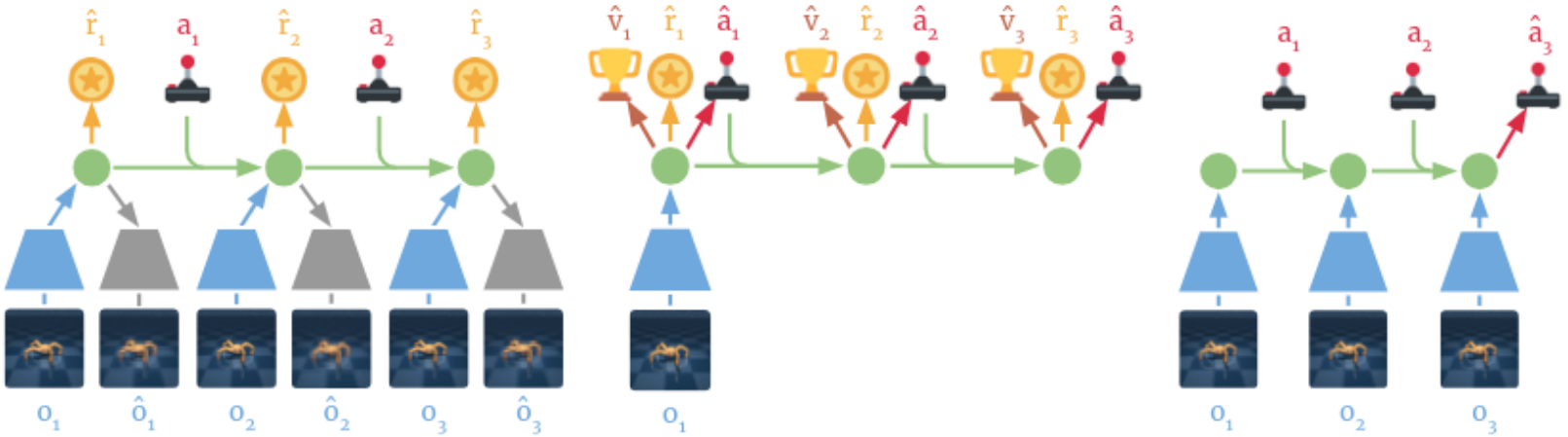
[7] https://arxiv.org/pdf/1912.01603

Agent Components

Dreamer operates through three interleaved processes:

1. **Dynamics Learning**: Learning the latent world model from a dataset of past experience.
2. **Behavior Learning**: Learning an action model (Actor) and value model (Critic) purely from imagined trajectories.
3. **Environment Interaction**: Executing the learned policy in the real world to collect new data.

Agent Components



(a) Learn dynamics from experience (b) Learn behavior in imagination (c) Act in the environment

Figure 3: Components of Dreamer. (a) From the dataset of past experience, the agent learns to encode observations and actions into compact latent states (●), for example via reconstruction, and predicts environment rewards (●). (b) In the compact latent space, Dreamer predicts state values (🏆) and actions (🕹) that maximize future value predictions by propagating gradients back through imagined trajectories. (c) The agent encodes the history of the episode to compute the current model state and predict the next action to execute in the environment. See Algorithm 1 for pseudo code of the agent.

Latent Dynamics Components

The world model consists of three neural network components:

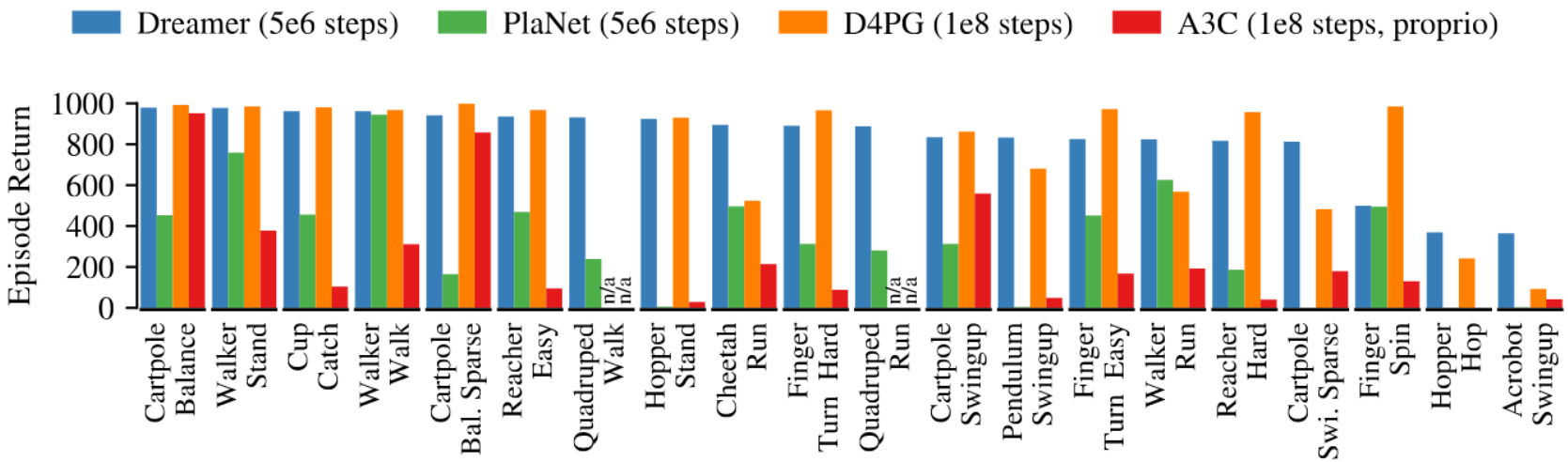
- Representation Model: Encodes observations and actions into Markovian model states s_t.
- Transition Model: Predicts future latent states without needing to generate actual images.
- Reward Model: Predicts potential rewards given the model states.

Why Imagination?

- **Speed**: Thousands of trajectories can be imagined in parallel due to the compact latent state footprint
- **Data Efficiency**: Dreamer can interpolate past experience and leverage analytic gradients for efficient optimization
- **Analytic Gradients**: Unlike derivative-free methods (e.g., World Models), Dreamer uses the world model as a differentiable environment

Empirical Success

- Dreamer was tested on 20 tasks in the DeepMind Control Suite[8]
- It exceeds previous model-based (PlaNet) and model-free (D4PG, A3C) agents in data-efficiency and final performance
- It is robust enough to use the same hyperparameters across different tasks

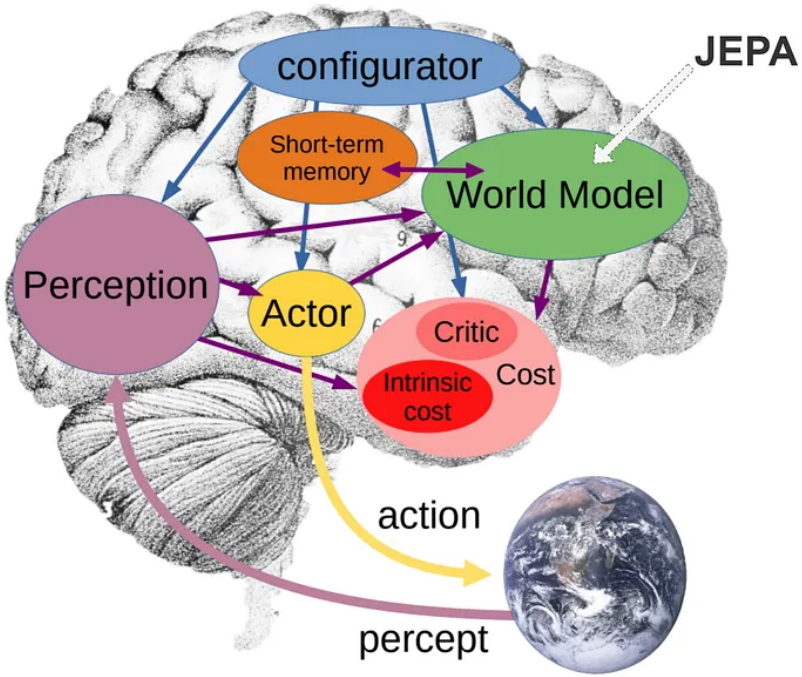


[8] https://arxiv.org/pdf/1801.00690

Outline

JEPA

- In 2022, Yann LeCun introduced the Joint Embedding Predictive Architecture (JEPA), a framework mirroring the human brain's structure
- It comprises a perception module that processes sensory data (i.e., an encoder), followed by a cognitive module (i.e., a predictor) that evaluates this information, effectively embodying the world model.



JEPA

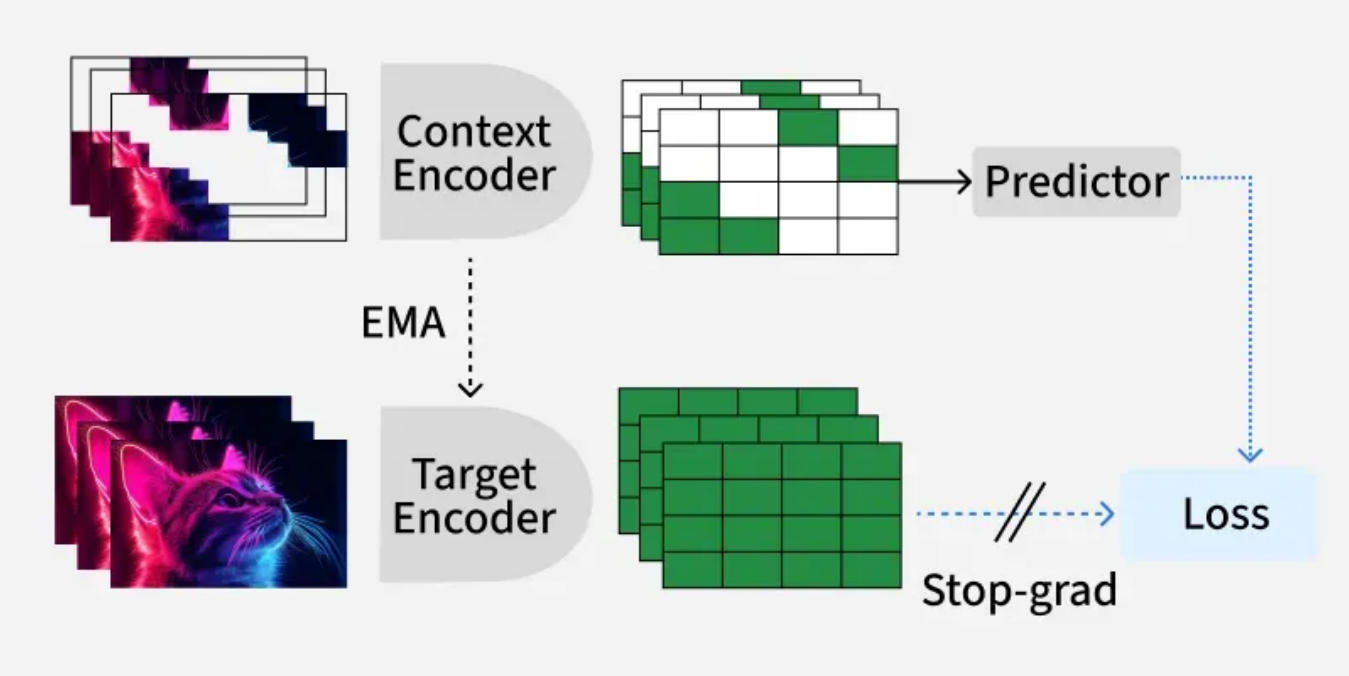
- This model allows the brain to assess actions and determine the most suitable responses for real-world applications.
- A key innovation of JEPA lies in its self-supervised learning paradigm, which enables the system to learn rich representations of the world without relying on extensive labeled data.
- Rather than predicting raw sensory inputs in pixel space, JEPA learns to predict abstract representations in a latent embedding space, making the learning process more efficient and robust.
- This approach allows the model to capture semantic relationships and causal structures in the data while avoiding the computational burden and potential pitfalls of pixel-level prediction
- In this framework, the world model is essential for understanding and representing the external world through self-supervised learning of latent variables, which capture key information while filtering out redundancies

JEPA Core Principles[9]

- Predictive Learning : Instead of predicting missing or future parts of the data directly (like guessing missing pixels), JEPA learns to predict useful patterns or features in a more abstract, hidden representation embedding (latent) space. This makes learning more efficient and less focused on irrelevant details.
- Joint Embedding : JEPA uses neural networks to convert both the visible part of the data (context) and the part it needs to predict (target) into the same shared space. This allows the model to "compare" the two in a meaningful way, helping it learn relationships and structure.
- Abstraction : By operating in the embedding space, JEPA encourages the model to focus on abstract, semantic features rather than low level details, which is beneficial for understanding and performing tasks across different domains or datasets (transfer learning) and downstream tasks.

[9] https://www.geeksforgeeks.org/artificial-intelligence/jepa/

JEPA Architecture[10]



[10] https://www.geeksforgeeks.org/artificial-intelligence/jepa/

JEPA Architecture[11]

- Context Encoder : Processes the observed part of the data (context) and produces a context embedding.
- Target Encoder : Processes the part of the data to be predicted (target) and produces a target embedding.
- Predictor (or Head) : Takes the context embedding and predicts the target embedding.
- Loss Function : The model is trained to minimize the distance between the predicted target embedding and the actual target embedding (often using contrastive or regression losses).

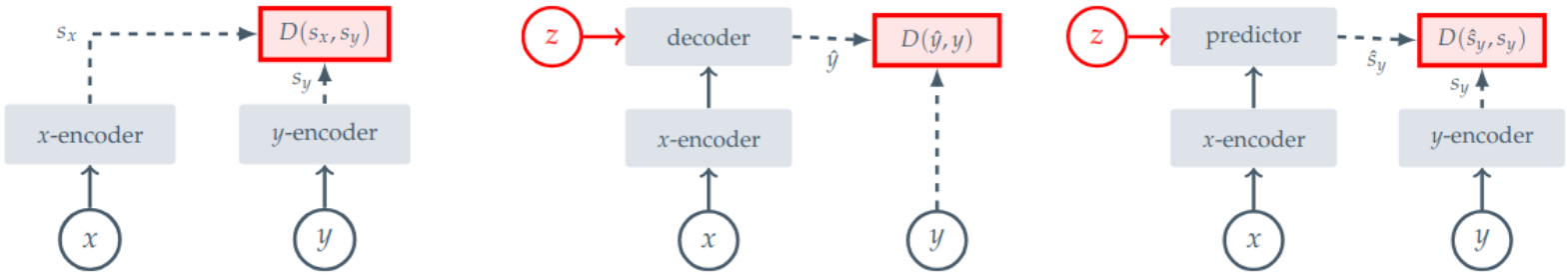
[11] https://www.geeksforgeeks.org/artificial-intelligence/jepa/

Addressing Issues of LLMs[12]

- JEPA learns internal, abstract models of the environment, enabling understanding and simulation of physical or semantic context beyond language.
- Integration of planning: Through explicit actor modules and latent variable modeling, JEPA supports both reactive and deliberate, planned reasoning addressing the lack of structured multi-step thinking in LLMs.
- Energy-based framework: JEPA uses energy based models that capture deep dependencies between inputs and outputs, rather than simple sequence prediction, enhancing its ability to represent ambiguous or uncertain futures.
- Handling uncertainty and multiple possibilities: Incorporating latent variables, JEPA naturally represents multiple plausible futures and manages prediction uncertainty, overcoming LLM 's limitations in dealing with ambiguous or multi-modal outcomes

[12] https://www.geeksforgeeks.org/artificial-intelligence/jepa/

JEPA vs. Generative Models



(a) **Joint-Embedding Architecture** (b) **Generative Architecture** (c) **Joint-Embedding Predictive Architecture**

Figure 2. Common architectures for self-supervised learning, in which the system learns to capture the relationships between its inputs. The objective is to assign a high energy (large scaler value) to incompatible inputs, and to assign a low energy (low scaler value) to compatible inputs. **(a)** Joint-Embedding Architectures learn to output similar embeddings for compatible inputs x, y and dissimilar embeddings for incompatible inputs. **(b)** Generative Architectures learn to directly reconstruct a signal y from a compatible signal x, using a decoder network that is conditioned on additional (possibly latent) variables z to facilitate reconstruction. **(c)** Joint-Embedding Predictive Architectures learn to predict the embeddings of a signal y from a compatible signal x, using a predictor network that is conditioned on additional (possibly latent) variables z to facilitate prediction.

I-JEPA Architecture

The system uses three main Vision Transformer (ViT) components:

Context Encoder: Processes a single visible "context" block of the image

Target Encoder: Produces the representations for "target" blocks. Its weights are an **EMA** of the context encoder

Predictor: A narrow ViT that takes context outputs and positional tokens to predict target representations

I-JEPA Architecture

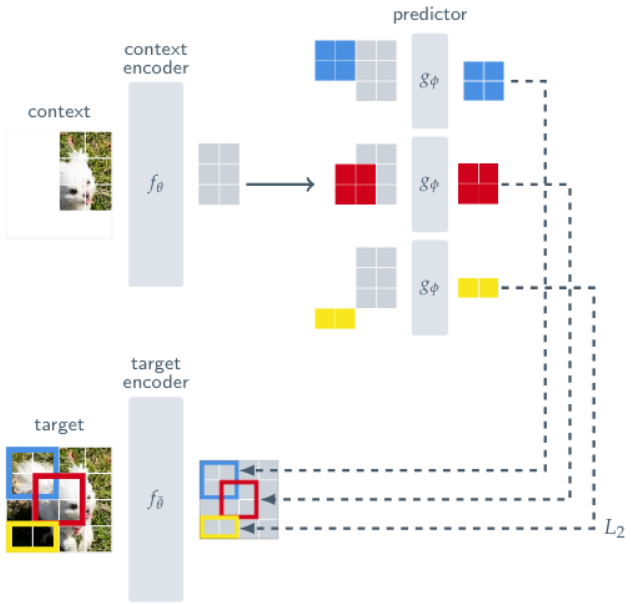


Figure 3. **I-JEPA.** The Image-based Joint-Embedding Predictive Architecture uses a single context block to predict the representations of various target blocks originating from the same image. The context encoder is a Vision Transformer (ViT), which only processes the visible context patches. The predictor is a narrow ViT that takes the context encoder output and, conditioned on positional tokens (shown in color), predicts the representations of a target block at a specific location. The target representations correspond to the outputs of the target-encoder, the weights of which are updated at each iteration via an exponential moving average of the context encoder weights.

Predicting Abstract Targets

- Targets are **not pixels**; they are patch-level representations from the target encoder
- By predicting in latent space, the model can eliminate unnecessary pixel-level details and focus on **semantic features**
- This abstraction is a core reason why I-JEPA outperforms pixel-reconstruction methods in linear probing

I-JEPA Masking Strategy

Figure 4. Examples of our context and target-masking strategy. Given an image, we randomly sample 4 target blocks with scale in the range $(0.15, 0.2)$ and aspect ratio in the range $(0.75, 1.5)$. Next, we randomly sample a context block with scale in the range $(0.85, 1.0)$ and remove any overlapping target blocks. Under this strategy, the target-blocks are relatively semantic, and the context-block is informative, yet sparse (efficient to process).

The Masking Strategy

The masking strategy is crucial for guiding the model toward semantic understanding:

- **Target Blocks:** Sampled with a "semantic" scale (0.15 to 0.2 of the image)
- **Context Block:** A large, informative, spatially distributed block (0.85 to 1.0 of the image)
- Overlapping regions between context and target are removed to ensure a non-trivial prediction task

Loss Function in Latent Space

- The loss is the **average L2 distance** between the predicted patch-level representations and the actual target representations
- No hand-crafted view augmentations (like color jittering or solarization) are used during pretraining
- This makes the architecture more generalizable to other modalities like audio or text

Visualizing Latent Predictions

- To understand what the predictor learns, the authors used a generative model to decode the latent predictions back into pixels.
- The predictor correctly captures **positional uncertainty** and high-level object parts (e.g., a bird's back or a car's top).
- It discards precise low-level details and background noise.

Visualizing Latent Predictions

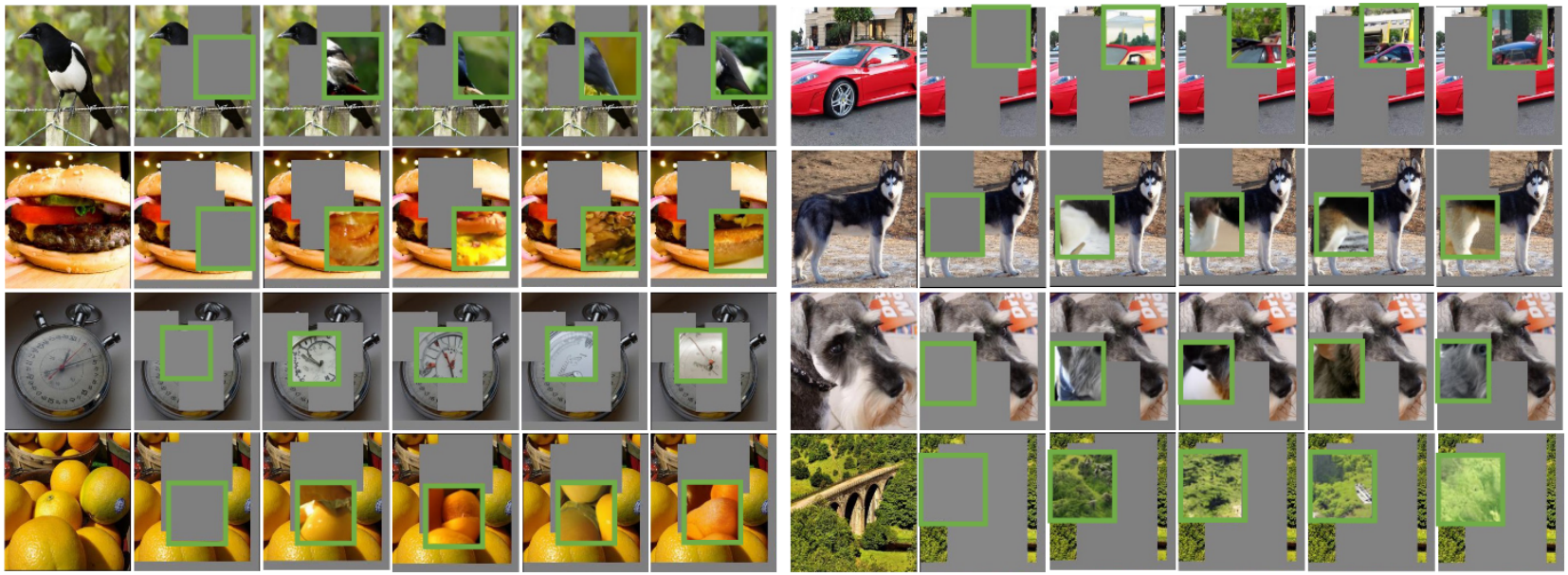


Figure 6. **Visualization of I-JEPA predictor representations**. For each image: first column contains the original image; second column contains the context image, which is processed by a pretrained I-JEPA ViT-H/14 encoder. Green bounding boxes in subsequent columns contain samples from a generative model decoding the output of the pretrained I-JEPA predictor, which is conditioned on positional mask tokens corresponding to the location of the green bounding box. Qualities that are common across samples represent information that contained is in the I-JEPA prediction. The I-JEPA predictor is correctly capturing positional uncertainty and producing high-level object parts with a correct pose (e.g., the back of the bird and top of a car). Qualities that vary across samples represent information that is not contained in the representation. In this case, the I-JEPA predictor discards the precise low-level details as well as background information.

Outline

Evolution: From Images to Video

- **V-JEPA** extends I-JEPA concept (Image-based JEPA) to the temporal dimension, learning from 1 million hours of unlabeled internet video
- Like I-JEPA, it avoids hand-crafted augmentations and pixel reconstruction, focusing on **predictable latent features**

The V-JEPA 2 Core Objective

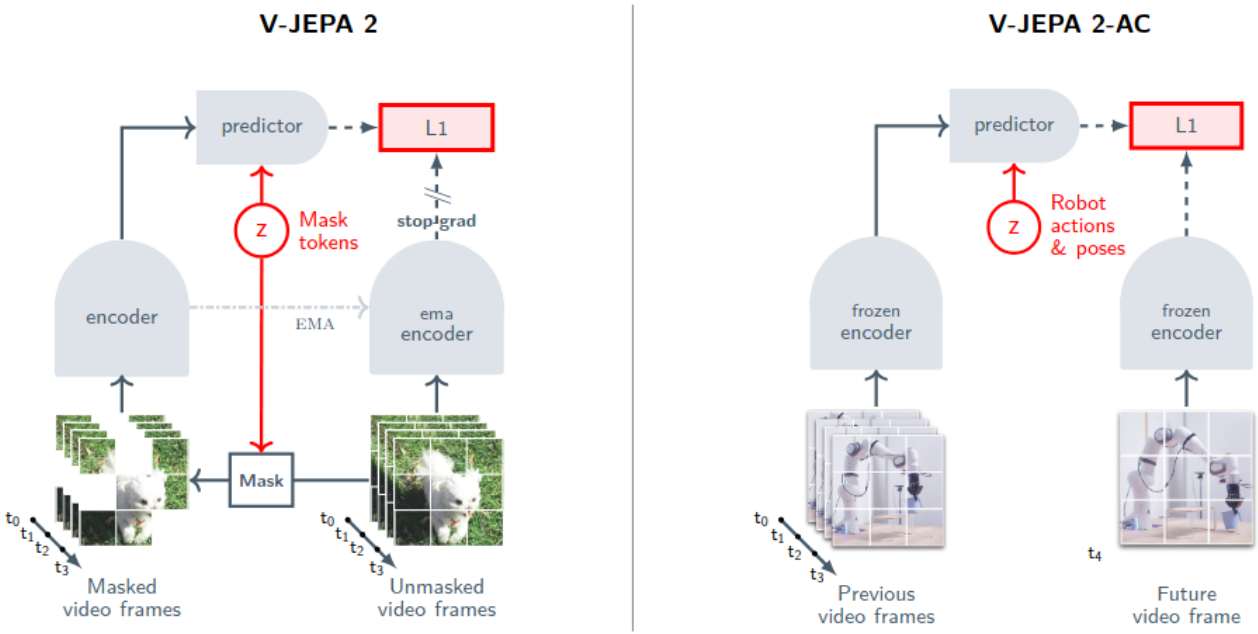


Figure 2 Multistage training. (Left) We first pretrain the V-JEPA 2 video encoder on internet-scale image and video data using a visual mask denoising objective (Bardes et al., 2024; Assran et al., 2023). A video clip is patchified into a sequence of tokens and a mask is applied by dropping a subset of the tokens. The encoder then processes the masked video sequence and outputs an embedding vector for each input token. Next, the outputs of the encoder are concatenated with a set of learnable mask tokens that specify the position of the masked patches, and subsequently processed by the predictor. The outputs of the predictor are then regressed to the prediction targets using an L1 loss. The prediction targets are computed by an ema-encoder, the weights of which are defined as an exponential moving average of the encoder weights. **(Right)** After pretraining, we freeze the video encoder and learn a new action-conditioned predictor, V-JEPA 2-AC, on top of the learned representation. We leverage an autoregressive feature prediction objective that involves predicting the representations of future video frames conditioned on past video frames, actions, and end-effector states. Our action-conditioned predictor uses a block-causal attention pattern such that each patch feature at a given time step can attend to the patch features, actions, and end-effector states from current and previous time steps.

Learning to Act: V-JEPA 2-AC

- After action-free pretraining, the model is fine-tuned on **robotic interaction data** (62 hours)
- It adds actions and end-effector states to the predictor's input using **block-causal attention**
- This turns the representation into a functional **Action-Conditioned World Model**

Planning in Latent Space

- The model plans by selecting actions that minimize the L1 distance between the **imagined future state** and a goal image.
- It generalizes **zero-shot** to Franka robot arms in new labs without ever seeing those environments.
- High success rates in complex tasks like **Pick-and-Place** using image sub-goals.

Planning in Latent Space

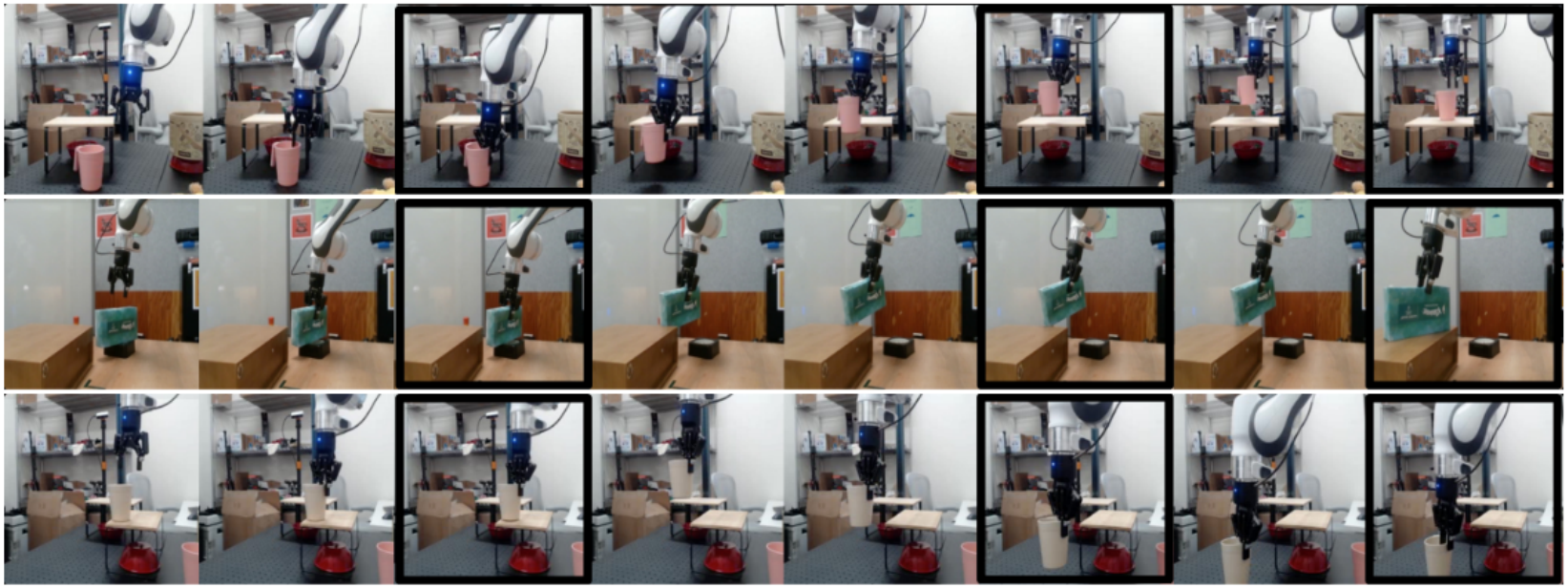


Figure 10 Pick-&-Place. Closed-loop robot execution of V-JEPA 2-AC for multi-goal pick-&-place tasks. Highlighted frames indicate when the model achieves a sub-goal and switches to the next goal. The first goal image shows the object being grasped, the second goal image shows the object in the vicinity of the desired location, and the third goal image shows the object placed in the desired position. The model first optimizes actions with respect to the first sub-goal for 4 time-steps before automatically switching to the second sub-goal for the next 10 time-steps, and finally the third goal for the last 4 time-steps. Robot actions are inferred through goal-conditioned planning. The V-JEPA 2-AC model is able to perform zero-shot pick-&-place tasks on two Franka arms in different labs, with various object configurations and cluttered environments.

Video Question-Answering

- V-JEPA 2 can be aligned with Large Language Models (e.g., Llama 3.1)
- It achieves SOTA results on benchmarks requiring **physical understanding** (PerceptionTest) and **temporal reasoning** (TempCompass)
- Proves that video encoders trained **without language supervision** can match or beat contrastive models like CLIP.

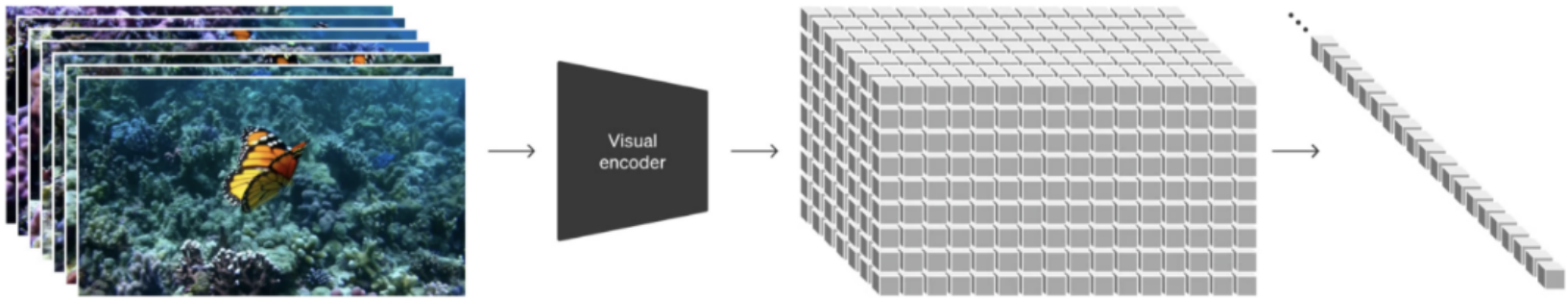
SORA

- Sora is a text-to-video generative AI model released by OpenAI in February 2024.
- Generates up to 1-minute long videos that exhibit a sense of narrative progression and visual consistency

The Building Blocks: Spacetime Latent Patches

- Sora transforms diverse visual data into a **unified representation** by "patchifying" video
- **Process:** Raw video is compressed into a lower-dimensional latent space, which is then decomposed into spacetime patches
- **Analogy:** These patches act like word tokens in LLMs, encapsulating both **spatial appearance and temporal dynamics**

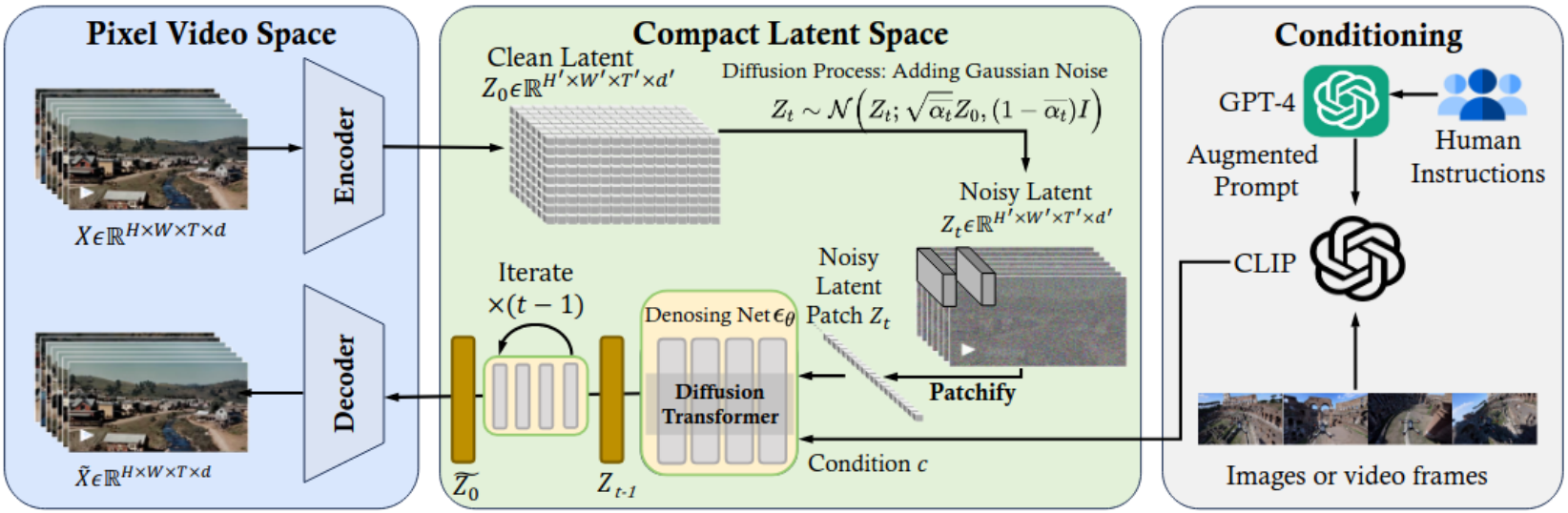
Turning videos into spacetime patches



Modeling Dynamics via Diffusion Transformers

- At its core, Sora uses a **Diffusion Transformer (DiT)** architecture
- **Mechanism:** Starting with a frame of visual noise, the transformer iteratively **denoises the representation** to construct the video
- This combines the generative power of diffusion with the scalability and long-range dependency handling of transformers

Overview of Sora framework



Emergent Physical Simulation

Scaling Sora has led to sophisticated behaviors not explicitly programmed:

- **3D Consistency:** Exhibits stable camera motion and perspective without explicit 3D modeling
- **Object Persistence:** Objects persist accurately even when occluded or leaving the frame
- **Digital Environments:** Can simulate complex digital worlds like Minecraft with visual fidelity

Generative vs. Predictive World Models

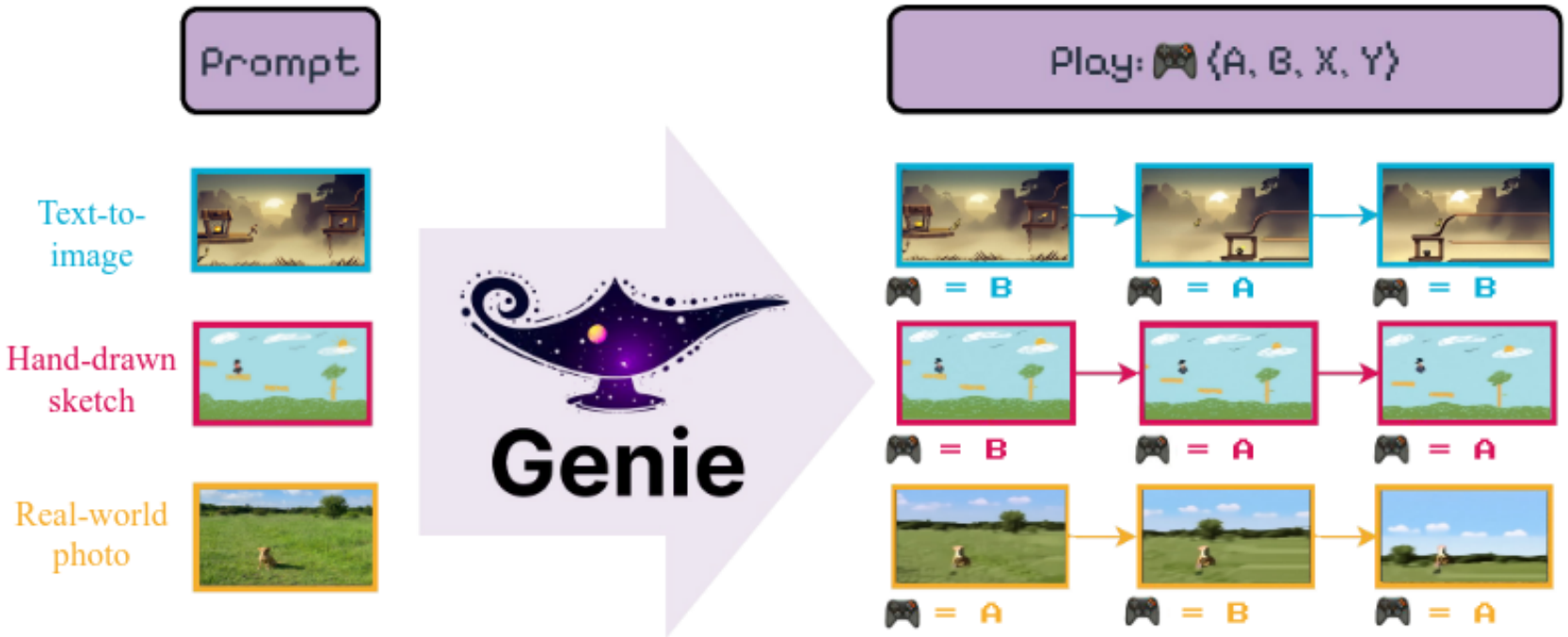
Sora (Generative): Learns by **pixel-level prediction** (reconstruction), which can waste capacity on unpredictable details like leaves on a tree

V-JEPA (Predictive): Focuses on **predictable latent features** (e.g., the trajectory of an object), ignoring noise for more semantic abstraction

Robotics: Video Prediction for Decision Making

- Generative models like Sora are being explored as **basis for robot planning**
- **Action Prediction:** Robots can "imagine" the outcomes of their actions in video format to decide the best path
- **Sim-to-Real:** Sora can generate diverse, highly realistic training scenarios for robots, mitigating real-world data scarcity
- Sora represents a milestone in Large Vision Models (LVMs), showing that scaling video data is a viable path to world simulation

Genie Example



Genie

- The first generative interactive environment trained in an **unsupervised manner** from unlabelled Internet videos
- At **11 Billion parameters**, Genie is considered a foundation world model
- Generates action-controllable virtual worlds from text, images, or even hand-drawn sketches

The Challenge: Acting Without Labels

- **Traditional RL Models:** Usually require ground-truth action labels (e.g., "move left," "jump") to learn dynamics.
- **Genie's Breakthrough:** It learns the "laws of the world" purely from 2D internet videos **without action labels** or domain-specific requirements
- It discovers the causal relationship between frames automatically during pretraining

Technical Framework

Similar to Sora's spacetime patches, Genie uses a tokenizer to compress high-dimensional raw video into a compact latent representation

Genie is comprised of three primary components that enable world simulation:

1. **Spatiotemporal Video Tokenizer:** Discretizes video into manageable "visual tokens."
2. **Latent Action Model:** Infers the underlying actions taken between frames.
3. **Autoregressive Dynamics Model:** Predicts the next state based on the current state and inferred action.

Discovering Control

- The **Latent Action Model** is the heart of Genie's interactivity
- It learns a **latent action space** by looking at pairs of frames and figuring out what "action" must have occurred to change the scene.
- This results in a scalable way to give users frame-by-frame control without ever seeing an actual controller input during training

Frame-by-Frame Interactivity

- Genie enables users to **act in the generated environments** in real-time
- Because the dynamics are autoregressive, each user input (latent action) triggers the world model to hallucinate the next logical frame.
- This transforms a passive video generation model into a **playable simulation**

From Sketches to Worlds

Genie can be initialized with diverse starting points:

- **Synthetic Images:** Created by other AI models (e.g., DALL-E).
- **Photographs:** Real-world images can become the first frame of a game.
- **Sketches:** Hand-drawn ideas can be animated into interactive environments.
- **Text:** Descriptions of settings and dynamics.

Path to Generalist AI

- Genie's learned latent action space facilitates training agents to **imitate behaviors from unseen videos**
- Since the model creates a "differentiable sandbox," AI agents can be trained inside Genie's dreams to perform tasks before being deployed in the real world.
- This aligns with the "World Model" vision of training agents in **simulated imagination**
- Genie bridges the gap between passive video generation and active environment simulation.

Outline

Extending the World Model Paradigm

- Previous world models (Ha & Schmidhuber, Dreamer) used generative models to "hallucinate" visual environments for RL agents.
- **RAP**[13] applies this same philosophy to **Language Models**.
- Instead of raw pixels, RAP treats natural language as the state space, where the LLM "imagines" the outcomes of its own reasoning steps.

[13] https://arxiv.org/abs/2305.14992

The Problem: Autoregressive "Instinct"

- Current LLMs (using Chain-of-Thought) reason instinctively in an autoregressive manner.
- They lack an internal world model to predict world states or simulate long-term outcomes.
- This makes them struggle with tasks humans find easy, such as generating complex action plans

RAP Overview

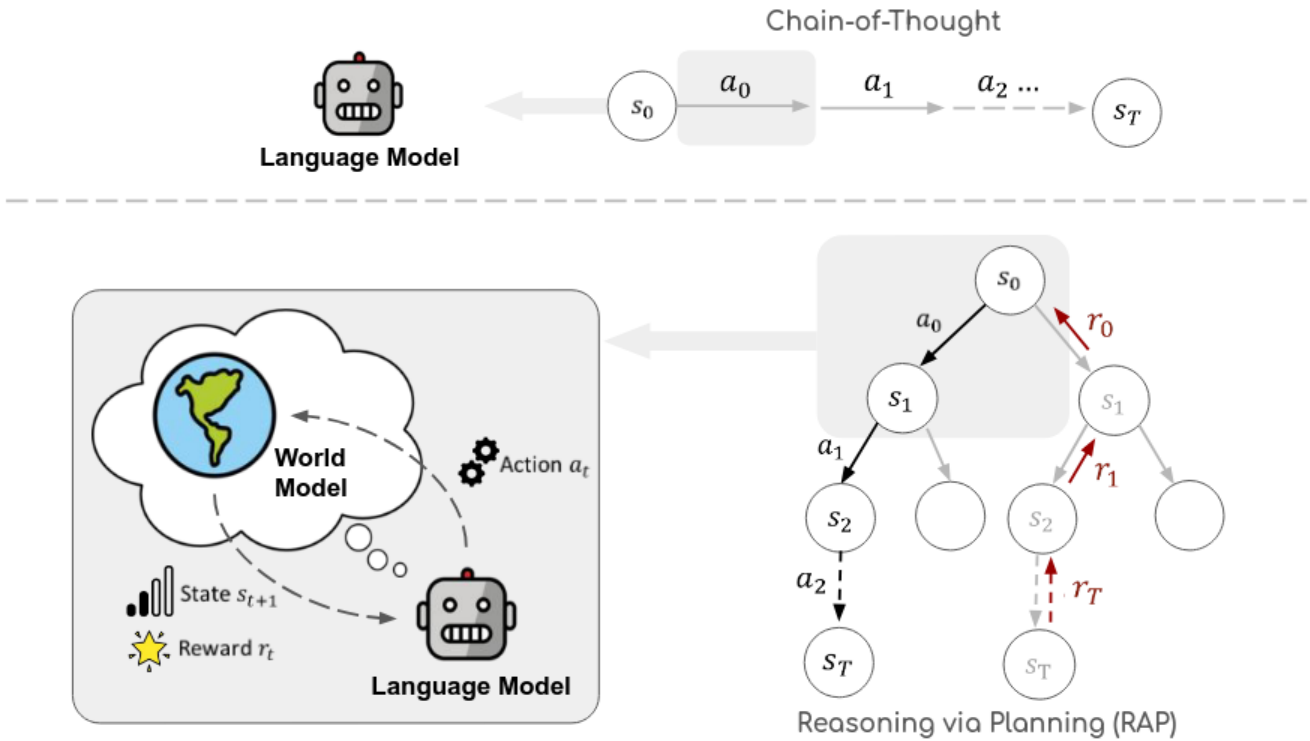


Figure 1: An overview of Reasoning via Planning (RAP). Compared with previous LLM reasoning methods like Chain-of-Thought (Wei et al., 2022), we explicitly model the world state from a world model (repurposed from the language model), and leverage advanced planning algorithms to solve the reasoning problems.

What is Reasoning via Planning (RAP)?

- RAP repurposes a single Large Language Model to act as both:
  1. **The Reasoning Agent:** Proposes potential actions/steps.
  2. **The World Model:** Predicts the resulting state of the world after an action.
- It uses a principled planning algorithm (**MCTS**) to explore the reasoning space strategically.

RAP for Plan Generation

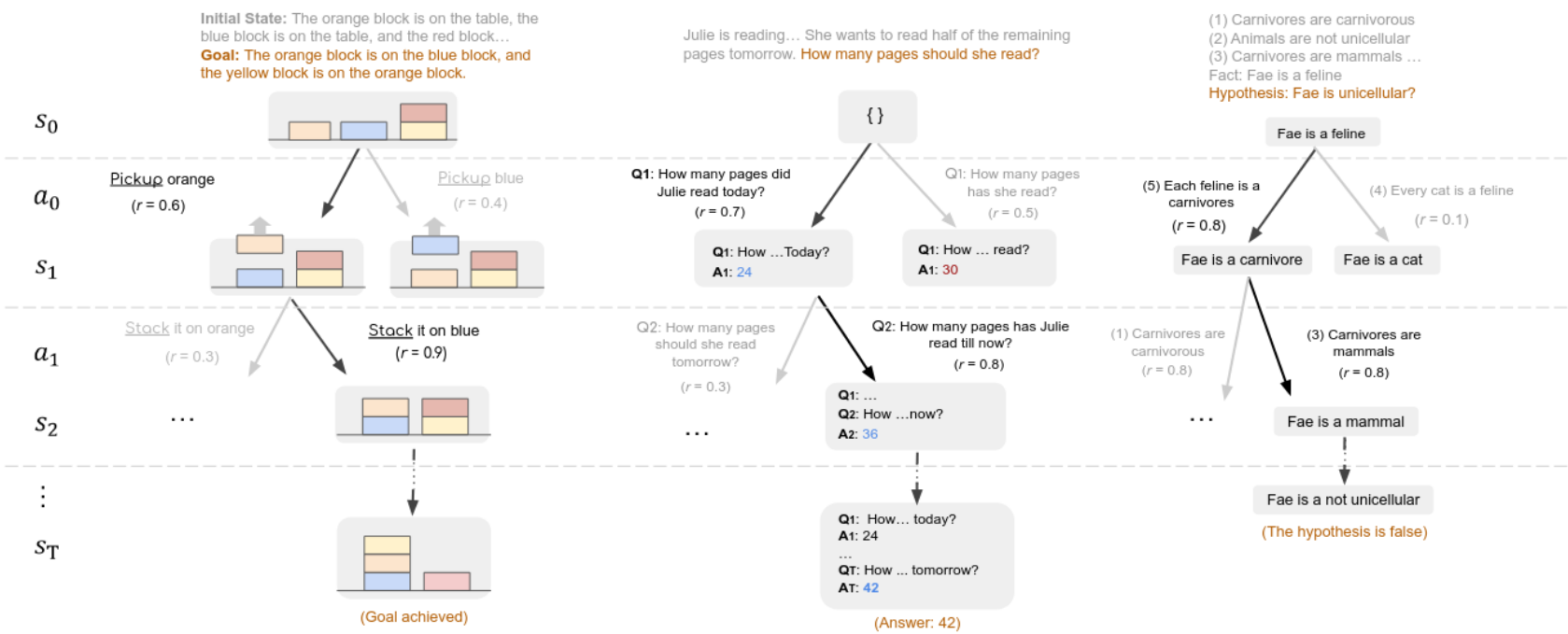


Figure 2: RAP for plan generation in Blocksworld (left), math reasoning in GSM8K (middle), and logical reasoning in PrOntoQA (right).

Defining the World in Text

RAP instantiates "states" and "actions" based on the task:

Blocksworld: State = block configuration; Action = moving a block.

Math Reasoning: State = known variable values; Action = a sub-question.

Logical Inference: State = current fact; Action = selecting a logical rule.

Reward Design

To guide the MCTS search, RAP uses several reward signals:

- **Action Likelihood:** The LLM's "instinctive" preference for a step.
- **State Confidence:** Consistency of predicted answers/states.
- **Self-Evaluation:** The LLM criticizing its own steps ("Is this correct?").
- **Task Heuristics:** Distance to the goal (e.g., blocks in correct position).

Strategic Exploration via MCTS

- RAP builds a **reasoning tree** where nodes are states and edges are actions.
- It uses Monte Carlo Tree Search to balance **Exploration** (trying new paths) and **Exploitation** (refining high-reward paths).
- Future rewards are backpropagated to update the value of current reasoning steps.

RAP Phases

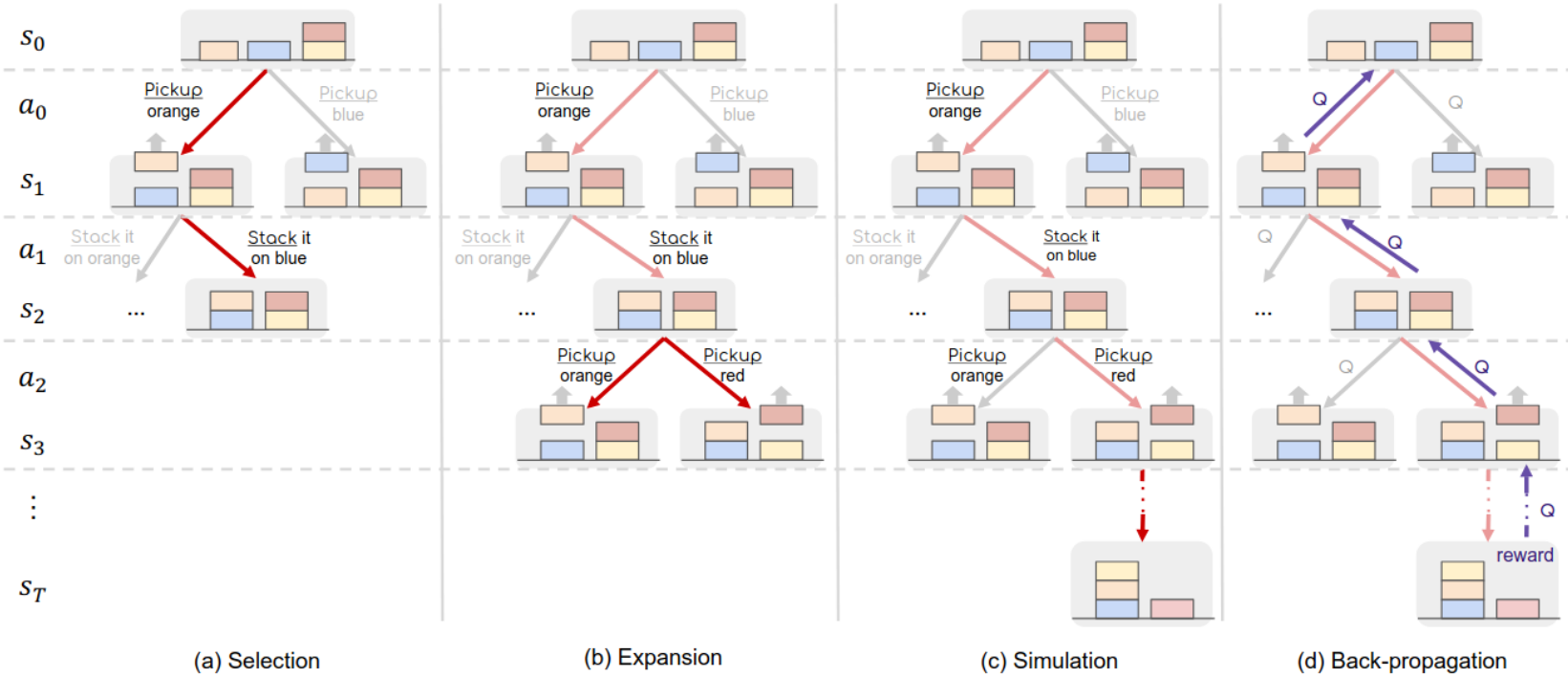


Figure 3: An illustration of the four phases in an iteration in MCTS planning (Section 3.3).

The Iterative Planning Cycle

Each MCTS iteration consists of four phases:

1. **Selection:** Choosing a promising path using the UCT algorithm.
2. **Expansion:** LLM agent samples potential reasoning steps.
3. **Simulation:** Using the LLM world model to "roll out" the future.
4. **Backpropagation:** Updating Q-values based on simulated outcomes.

Backtracking

- Unlike standard CoT, which fails if a single step is wrong, RAP can **backtrack**.
- If a reasoning chain hits a dead end (low reward), MCTS directs the search toward better alternatives.
- This allows a model like LLaMA-33B to outperform GPT-4 on complex planning tasks.

CoT vs RAP

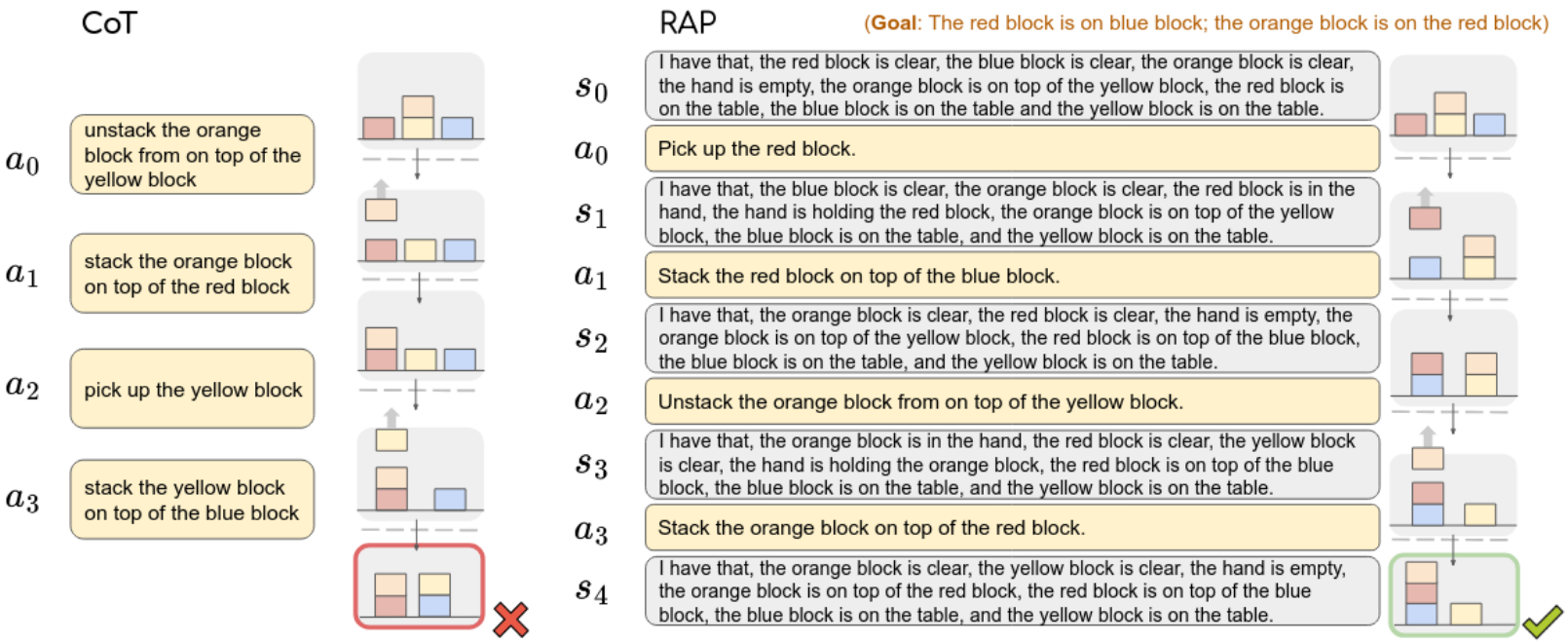


Figure 4: Comparing reasoning traces in Blocksworld from CoT (left) and RAP (right).